

17 · Resumen ejecutivo

Documentación de sistema

Versión analizada: **0.1.1+2**

Commit: **a7ebf2c**

Fecha del documento: **2026-08-27**

Documento generado desde el Markdown de **docs/system-documentation/** con **tool/build_system_documentation_pdf.py**. El Markdown es la fuente: no editar este PDF.

Contenido

1. Qué es
2. Qué necesidad cubre
3. Quién lo utiliza
4. Capacidades principales
5. Tecnologías
6. Arquitectura, en una idea
7. Estado actual
8. Fortalezas
9. Riesgos
10. Oportunidades de mejora
11. Próximos pasos recomendados
12. Conclusión

Qué es

Una aplicación móvil Android que **inspecciona códigos QR antes de abrirlos**. En lugar de llevar a la persona al destino y avisarle después, muestra qué hay dentro del código, explica qué señales de riesgo observó y solo entonces ofrece —o niega— una acción.

Todo el análisis ocurre en el teléfono. No hay servidor, cuentas, publicidad ni estadísticas de uso.

Qué necesidad cubre

Un QR es una instrucción que nadie puede leer a simple vista. Esa opacidad es justo lo que aprovecha el fraude: una pegatina sobre el QR de un parquímetro, un cartel con un cobro falso, un mensaje con un enlace que imita al del banco. Los lectores habituales del teléfono abren el destino primero y explican después.

Este producto invierte el orden. Su promesa es acotada y verificable: **reduce la opacidad**. No promete detectar fraude ni certificar que un sitio sea seguro, y lo dice explícitamente cada vez que no encuentra nada.

Quién lo utiliza

Perfil	Uso
Persona ante un QR dudoso	Verlo antes de abrirlo
Equipo de seguridad o soporte	Documentar un caso con evidencia compartible
Operación de almacén	Contar inventario leyendo códigos de producto
Organización con marca propia	Detectar su nombre usado fuera de sus dominios

Capacidades principales

- lectura con cámara, desde imágenes o desde un PDF por lotes;
- interpretación de 17 familias de contenido: enlaces, Wi-Fi, contactos, eventos, OTP, pagos, criptomonedas, códigos de producto, documentos;
- **26 comprobaciones locales** que producen hechos con su evidencia;
- separación explícita entre hecho observado e hipótesis;
- decisión de acción en cuatro niveles, con bloqueo de direcciones ambiguas;
- **evidencia exportable** que no incluye el enlace ni el secreto;
- historial e inventario cifrados en el dispositivo;
- generador de códigos y centro de recuperación de datos.

Tecnologías

Flutter 3.44.7 y Dart. Base de datos embebida Sembast, cifrado AES-256-GCM con la llave en el almacén seguro del sistema. Integración continua en GitHub Actions. Licencia MIT.

Escala del código: 65 archivos fuente, unas 9 600 líneas, 88 casos de prueba, 13 herramientas de automatización, 36 documentos.

Arquitectura, en una idea

El motor de análisis **no depende de Flutter**: recibe un texto y devuelve un objeto. Esa decisión permite probarlo sin dispositivo, comparar resultados entre versiones y, en el futuro, reutilizarlo desde otro producto. Todo lo demás —pantalla, base de datos, exportación— son proyecciones de ese núcleo.

Estado actual

Aspecto	Estado
Código fuente	Completo y coherente; verificadores automáticos en verde
Integración continua	Verde: análisis estricto, 87 pruebas y tres compilaciones
Distribución Android	Publicada : APK con SHA-256 y atestación de procedencia
Firma de tienda	Pendiente : se firma con la clave de depuración de Flutter
iOS	Compila para simulador; sin paquete instalable ni firma
Validación en dispositivo físico	Pendiente : la matriz no tiene ninguna fila registrada
Auditoría de seguridad independiente	No realizada

La versión 0.1.1 corrige tres fallos de interacción reportados en uso real: una lectura conseguida no se distinguía de que no ocurriera nada; un código legible pero lejano no se leía; y volver a apuntar al mismo código no producía ningún efecto. Las tres causas están documentadas con su corrección.

Fortalezas

1. **La promesa es verificable.** Cada afirmación del producto apunta a código, prueba o documento. Lo pendiente se declara como pendiente.
2. **Privacidad real, no declarativa.** Sin red no hay nada que auditar: la ausencia de telemetría es estructural.
3. **Disciplina de datos poco común.** Un registro ilegible se aísla en vez de borrarse; una llave ausente nunca se recrea; una migración verifica antes de borrar el origen; un respaldo importado no puede imponer un veredicto.
4. **Contratos ejecutables.** Dos verificadores fallan si una regla queda huérfana entre el motor, el esquema, los textos y la documentación. Las decisiones documentales son gates, no buenas intenciones.
5. **Honestidad como control de calidad.** La aplicación se niega a decir «seguro», y hay una prueba que impide quitar esa frase.

Riesgos

Riesgo	Severidad	Consecuencia
Los 12 casos de regresión existen pero ninguna prueba los ejecuta	Alta	Un cambio de regla puede pasar el gate sin que se note
La migración de datos heredados no tiene prueba propia	Alta	Es el procedimiento con más riesgo de pérdida
Los contactos de una vCard se guardan sin marcarse sensibles	Alta	Datos personales de terceros conservados sin decisión explícita

La corrección de lectura no se ha probado en un teléfono real	Alta	La mejora podría no funcionar, o calentar la gama baja
La evidencia lleva checksum, no firma	Alta	No acredita autoría frente a un adversario
Sin validación en dispositivos físicos	Alta	Cámara, biometría y almacenamiento sin confirmar
APK firmado con clave de depuración	Alta	No apto para tienda; complica la actualización
Dos dependencias ancladas por un conflicto	Media	Un aviso de seguridad exigiría resolverlo antes
Un solo mantenedor	Media	Factor bus de uno

Ninguno es crítico. Todos están documentados en [15-risks-and-technical-debt.md](#) con su recomendación.

Oportunidades de mejora

Coste bajo, valor alto

1. Ejecutar los 12 fixtures contra el motor en una prueba automática.
2. Cubrir la migración de datos heredados con pruebas.
3. Decidir explícitamente qué hacer con los contactos.

Coste medio

4. Ejecutar la matriz de dispositivos en al menos dos teléfonos.
5. Unificar el inventario con la abstracción del escáner, para no tener que corregir dos veces cada fallo de cámara.
6. Umbral de cobertura en CI.

Decisiones de producto

7. Firma comercial y publicación en tienda.
8. Firma criptográfica de la evidencia.
9. Completar la traducción para abrir el producto a más idiomas.
10. Pantalla de administración de la política de marcas, que hoy solo existe como API.

Próximos pasos recomendados

Plazo	Acción	Por qué
Inmediato	Prueba de los fixtures y de la migración	Cierra las dos brechas de mayor riesgo con esfuerzo bajo
Inmediato	Probar la corrección de lectura en dos teléfonos	Confirma o refuta la mejora que motivó la versión
Corto	Decisión sobre los contactos	Es una cuestión de privacidad, no técnica
Corto	Escaneo de vulnerabilidades sobre el SBOM	El inventario ya existe; falta usarlo

Medio	Matriz de dispositivos completa	Requisito declarado antes de una tienda
Medio	Firma comercial y App Bundle	Habilita la distribución masiva
Largo	Auditoría independiente	Necesaria antes de presentar el producto como herramienta de seguridad

Conclusión

El repositorio está por encima de la media en algo que suele fallar: **la distancia entre lo que un producto dice de sí mismo y lo que hace**. Aquí esa distancia es corta y está medida. Las carencias son de *validación* —dispositivos físicos, firma, auditoría— más que de *construcción*.

Para un uso profesional serio, el orden recomendado es: cerrar las dos brechas de prueba, validar en hardware, y solo entonces plantear la distribución en tienda.